

A Perceptive Markowitz Model

S. Lin

*M.S. Aeronautics and Astronautics
AA 222 - Stanford University*

Abstract - This project presents an extension of the classical Markowitz portfolio optimization framework by incorporating dynamic market perception through a genetic algorithm-based search strategy. The proposed model jointly optimizes asset weightings and holding duration to maximize the Sharpe ratio, adapting to non-stationary windows of relevant financial data. The model supports user-defined asset lineups, toggles for short-selling, and employs a genetic approach. Results indicate that the algorithm is capable of identifying high-performing portfolio configurations within the defined historical window. However, when said portfolio is compared against random portfolios and evaluated under observed market trajectories, performance remains indeterminate overall. Notably, more consistent patterns of out-performance were observed for certain asset categories, suggesting that the model's effectiveness may be contingent on the categorical makeup of the lineup. This model may be further improved using Surrogate methods or optimizing on multiple objectives.

I. Introduction

Quantitative finance uses math, statistics, and algorithms to guide financial decision-making, bringing structure to the uncertainty of investing. While professional quants often work at high frequencies, high stakes, and secretive algorithms, this project was born from a softer motivation: to help the casual investor make smarter choices without giving up their personal touch.

The Perceptive Markowitz Model is designed for low-frequency investors who want to supplement their intuition with rigorous, adaptive optimization. It provides an efficient way to explore asset allocation using evolutionary techniques, blending modern portfolio theory with a dynamic understanding of time.

Given a lineup of assets ranging from blue-chip stocks to crypto, ETFs, or bonds, the model outputs two key insights:

- 1) a suggested allocation of funds across the selected assets
- 2) a recommended holding time based on recent market conditions.

By evolving both portfolio weights and historical lookback periods, the model adapts to shifting expected returns and volatility rather than use standard periodic trackers, empowering investors with tailored, perceptive strategies that consider the holding time of a portfolio as well as its composition.

II. A Perceptive Markowitz Model

A. Problem Statement

The Markowitz model is the foundation of Modern Portfolio Theory (MPT). This model is fundamentally a mean-variance analysis strategy that seeks to find the most efficient portfolio given a set of assets. Colloquially, the Markowitz model considers the expected returns and covariances of a set of assets and optimizes the weights of allocation of funds across said assets [1].

$$\sigma_p = \frac{1}{2} \omega \Sigma^T \omega \quad (1)$$

$$R_p = \omega^T r \quad (2)$$

$$\text{subject to } \omega^T \mathbf{e} = 1 \quad (3)$$

$\omega \in \mathbb{R}^n$ is defined as an n dimensional vector of asset weights. ω is typically the design variable. Eq 1 & 2 calculate the portfolio volatility σ_p and expected returns R_p respectively. $\Sigma \in \mathbb{R}^{n \times n}$ and $r \in \mathbb{R}^n$ are typically defined by historical data over a fixed timespan (usually 1 year). Importantly Σ is almost certainly positive definite. The constraint imposed by Eq 3 simply states that the elements of ω must sum to 1, where $\mathbf{e} \in \mathbb{R}^n$ is just an n dimensional vector of 1s.

A classic objective function to the Markowitz model is the Sharpe ratio.

$$S_a = \frac{R_p - R_f}{\sigma_p} \quad (4)$$

Intuitively, maximizing this ratio will maximize expected returns while minimizing volatility (volatility is always positive). R_f here is the current market risk-free rate, which is defined as the current rate of return on a risk-free asset such as a government bond. However, since it is a scalar independent of our design variable, we can safely drop this term in our effective objective function.

The Markowitz optimization problem itself is sufficient for a classic optimization problem, however in order to add flavor and levels of optimization to the project, I seek to improve the Markowitz model by adding a design variable: time. r and Σ can be thought of as functions of a look-back range T [2].

$$\mu_i(T) = \sum_{t=0}^T \ln \frac{P_{C-t}}{P_{C-t-1}} \quad (5)$$

$$\sigma_i(T) = \sqrt{\frac{T}{T-1} \sum_{t=0}^T \left(\ln \frac{P_{C-t}}{P_{C-t-1}} - \frac{1}{T} \mu_i(T) \right)^2} \quad (6)$$

$$r_i(T) = \exp \left(\left(\mu_i(T) - \frac{1}{2} \sigma_i(T)^2 \right) T \right) \quad (7)$$

$\mu_i(T)$ & $\sigma_i(T)$ are the drift and volatility respectively of asset i calculated on look-back range T . P_j is the closing asset price of day j with P_C being the current price. $r_i(T)$ is the expected return of asset i on holding time T calculated by $\mu_i(T)$ and $\sigma_i(T)$. Using the values of r_i and σ_i for each asset $i \in (1, n)$, Σ and r can be constructed for Eqs 1 & 2, and hence the Sharpe ratio can be evaluated after finding an optimal ω .

In summary, the objective Sharpe ratio is now a function on design variables ω (asset weights) & T (holding time), giving us a novel optimization problem by Eq 8.

$$w, T = \arg \min_{w \in \mathbb{R}^n, T \in \mathbb{R}} S_a(\omega, T) \quad (8)$$

B. Data Selection & Tailoring

To generate realistic and testable investment strategies, I sourced historical asset data using the yfinance Python API, which provides convenient access to Yahoo Finance's database. For each asset, I downloaded the most recent 10 years of historical price data at daily resolution.

Assets could be drawn from a variety of categories, including:

- Blue-chip equities (e.g., AAPL, MSFT)
- Exchange-traded funds (ETFs)
- Bonds or bond-like proxies
- Penny stocks and small-caps
- Cryptocurrencies

We used **only the adjusted closing prices** in our analysis. This is a conventional choice in low-frequency investing, as closing prices are stable daily reference points and incorporate adjustments for splits and dividends.

To prepare the data, all selected asset time series were **aligned and trimmed** to ensure matching timestamps across tickers. Since financial assets often have inconsistent trading histories or missing data, we intersected the available date ranges and excluded periods without full coverage. This ensured that all return and covariance computations were made using synchronized data windows.

A critical design choice was to optimize each portfolio **based on a fixed point in historical time**—specifically, the midpoint of the 10-year data window (typically 5 years in the past). This was done to simulate realistic investment decision-making: the model is trained on data available *up to* the midpoint, and its performance is then evaluated on the unseen data that follows. This approach avoids data leakage while allowing immediate evaluation of the strategy's efficacy, rather than waiting out the holding period in real time. It also provides a bootstrap-style mechanism for empirical validation across a wide range of synthetic training windows.

C. Methods and Algorithms

1. Why a Genetic Algorithm?

Genetic algorithms (GAs) are population-based optimization heuristics inspired by biological evolution. Though their mechanics—selection, crossover, mutation—are well-established, the key question is why such a method suits this particular portfolio problem.

Portfolio optimization in this setting is a highly stochastic task. The search space includes a potentially volatile, nonlinear, and discontinuous fitness surfaces due to the nature of financial return distributions.

Traditional gradient-based optimization techniques are poorly suited to this landscape. Our model relies on computed statistics (mean, variance, Sharpe ratio) derived from empirical price histories—quantities which are not analytically differentiable with respect to weights or lookback period.

In light of this, a genetic algorithm was the natural choice. It enables efficient search across both continuous (weights) and discrete (holding time) dimensions, without requiring smoothness or convexity in the objective function. Moreover, evolutionary strategies offer built-in robustness against local optima—a vital property given the potentially extreme granularity and unpredictability of the financial environment I model.

Importantly, our implementation deliberately emphasizes exploration. Rather than prematurely converging to narrow and all-too-common local optima, I encouraged broad coverage of the search space through mutation-heavy strategies and noise-tolerant fitness evaluation. This is aligned with the model’s intended use case: navigating noisy, uncertain data where generality and adaptability are more valuable than brittle precision.

Each genome in the population encodes two components: a vector of portfolio weights (which must be non-negative and sum to one) and a holding time parameter (a positive integer corresponding to the number of days used to compute past returns and covariances).

2. Mutation

Mutation is the dominant driver of exploration in this model. Each genome is subject to mutation with a fixed mutation rate, which determines the probability that any given individual is altered.

To preserve feasibility:

- Weight mutations must result in a valid probability vector (non-negative entries summing to 1).
- Holding time mutations must remain within a user-defined valid range of lookback periods.

Several mutation methods were implemented:

- **Gaussian perturbation** — Each weight is adjusted by a small random noise value, followed by normalization. Ultimately not used, as it lacked sufficient exploratory power.
- **Weight swap** — Two weights are randomly selected and exchanged. More impactful than Gaussian noise, but still insufficiently diverse for highly volatile asset spaces.
- **Dirichlet resampling (applied)** — The entire weight vector is regenerated by sampling from a Dirichlet distribution, ensuring feasibility while producing high-variance, high-diversity offspring. This method proved most effective for exploration.
- **Random holding time resample (applied)** — The holding time is reassigned to a random value within the feasible range.

In this application, mutation is the primary evolutionary force. Its design and intensity largely determine the algorithm’s ability to discover performant strategies, overshadowing the role of both selection and crossover.

3. Selection

Selection mechanisms determine which individuals are chosen to reproduce. Several methods were implemented:

- **Truncation** — The top-performing individuals are selected directly.
- **Tournament** — Random subsets of individuals compete, and winners proceed to the next generation.
- **Roulette wheel (applied)** — Individuals are selected probabilistically based on fitness. This method was ultimately chosen for its simplicity and lack of hyperparameters.

While selection affects population pressure, my experiments showed that the choice of selection mechanism had minimal impact relative to the mutation strategy. Thus, I favored the least parameter-sensitive method.

4. Crossover

Crossover played a minimal role in this algorithm. The implemented method was:

- **Interpolated weight crossover and uniform time choice** — Given two parent genomes, child weights are generated via linear interpolation (with a random mixing coefficient), while the holding time is randomly chosen from one of the parents.

No crossover hyperparameters were introduced, and feasibility of both weights and time was strictly maintained. However, crossover had little influence compared to mutation and was retained primarily for structural completeness.

5. Elitism

To prevent regression in fitness, a small fraction of the top individuals in each generation was passed directly to the next. This technique, known as **elitism**, introduces a subtle but crucial *greed factor*, ensuring that high-performing solutions are never lost due to stochastic variation in selection or mutation.

6. Summary of Evolutionary Strategy

This algorithm's behavior is primarily defined by two design choices:

- 1) **Highly exploratory mutation methods**, particularly Dirichlet resampling and randomized holding times, allow the model to remain robust in the face of volatile and noisy financial data.
- 2) **Nonzero elitism** introduces a stabilizing greed component, allowing consistently high-performing individuals to persist across generations.

Together, these two elements—exploration and greed—capture the essence of effective portfolio evolution in uncertain, nonconvex environments.

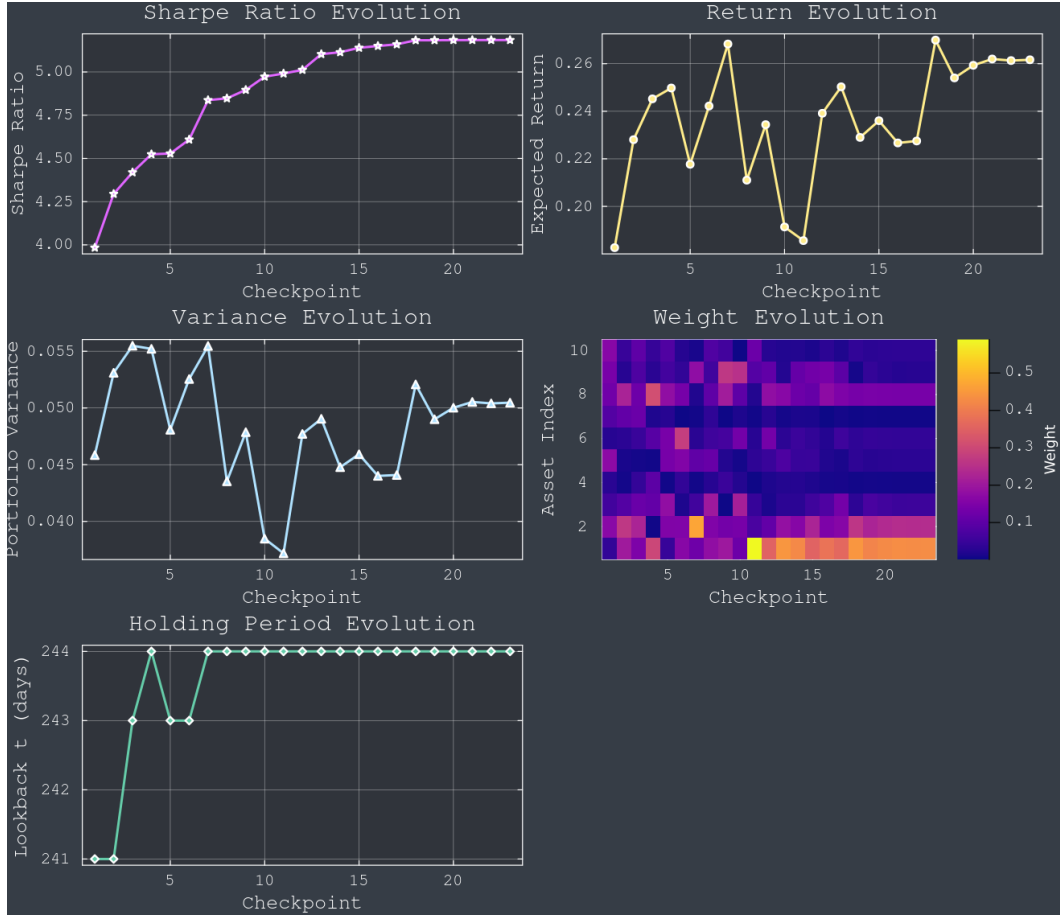


Fig. 1 Evolutionary Progression of Selected Genetic Algorithm.

7. Convergence Criteria

It was observed that the best-performing individual (by Sharpe ratio) often remained unchanged for many generations. However, this stagnation did not always imply true convergence; in many cases, it simply reflected the need for further mutation cycles to escape local optima or uncover superior strategies.

To balance efficiency and thoroughness, we defined convergence in terms of a **stagnation threshold**: the number of consecutive generations during which the best Sharpe ratio in the population does not improve. Once this threshold is reached, the algorithm halts.

Figure 1 displays the evolutionary progression of both the design variables and the objective Sharpe function (as well as its components in returns and variance). As expected, the objective Sharpe ratio monotonically increases and converges. However, the chaos inherently behind the visually smooth Sharpe Ratio evolution can be clearly seen in the stochasticity of the weight vector evolution towards the beginning as well as the resulting returns and variances. It is important to note that despite being presented as linearly-spaced, changes in plotted values did not occur equally-spaced. Only unique changes that reset the stagnation threshold were plotted.

D. Results

1. Single-Asset-Lineup

The following results were evaluated from a single, unique lineup of assets (10 unnamed bonds) as an example of the performance of my Perceptive Markowitz Model. In-window comparison displays how well the suggested portfolio compares to random portfolios evaluated within the provided domain of financial data. Out-of-Window prediction displays how well the suggested portfolio compares to random portfolios evaluated on the actual observed future trajectories. In other words, In-window comparison seeks to answer "how well did this optimization algorithm work based on what it was given" while Out-of-Window prediction seeks to answer "how effectively does this algorithm's output make a good financial decision"? As shown in Figure 2, the the genetic algorithm does a good job of optimizing

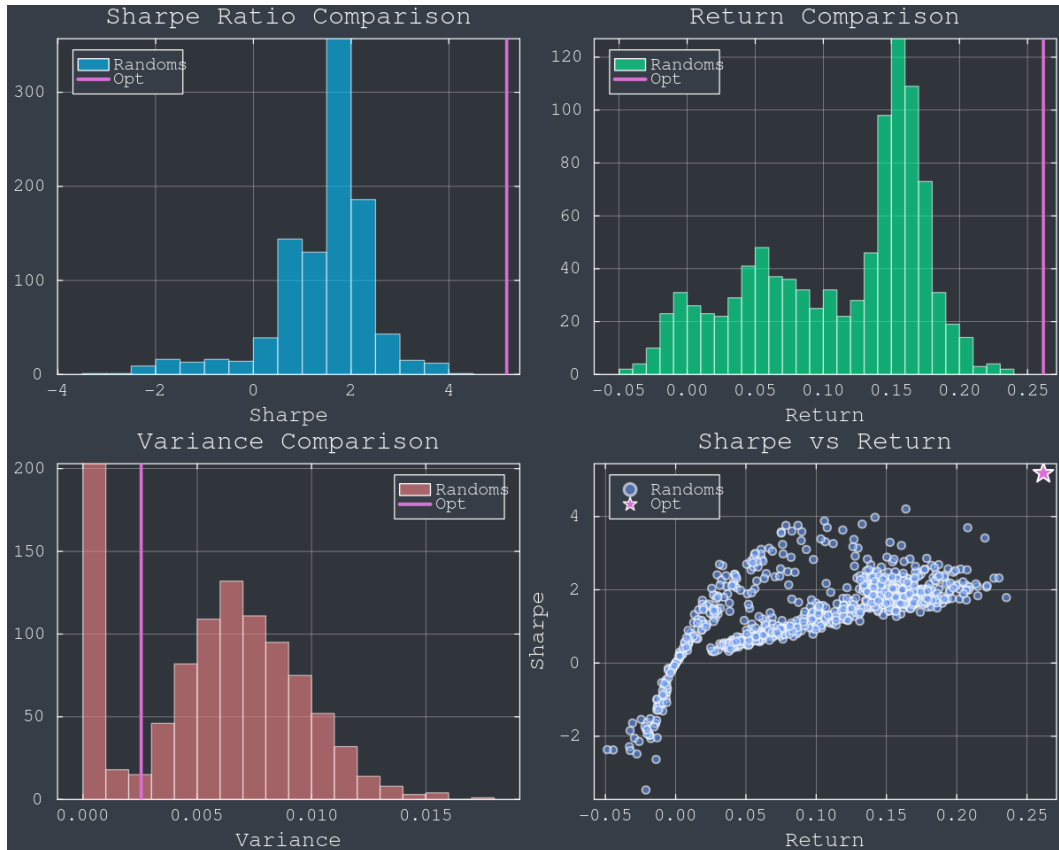


Fig. 2 Results for in-window comparison

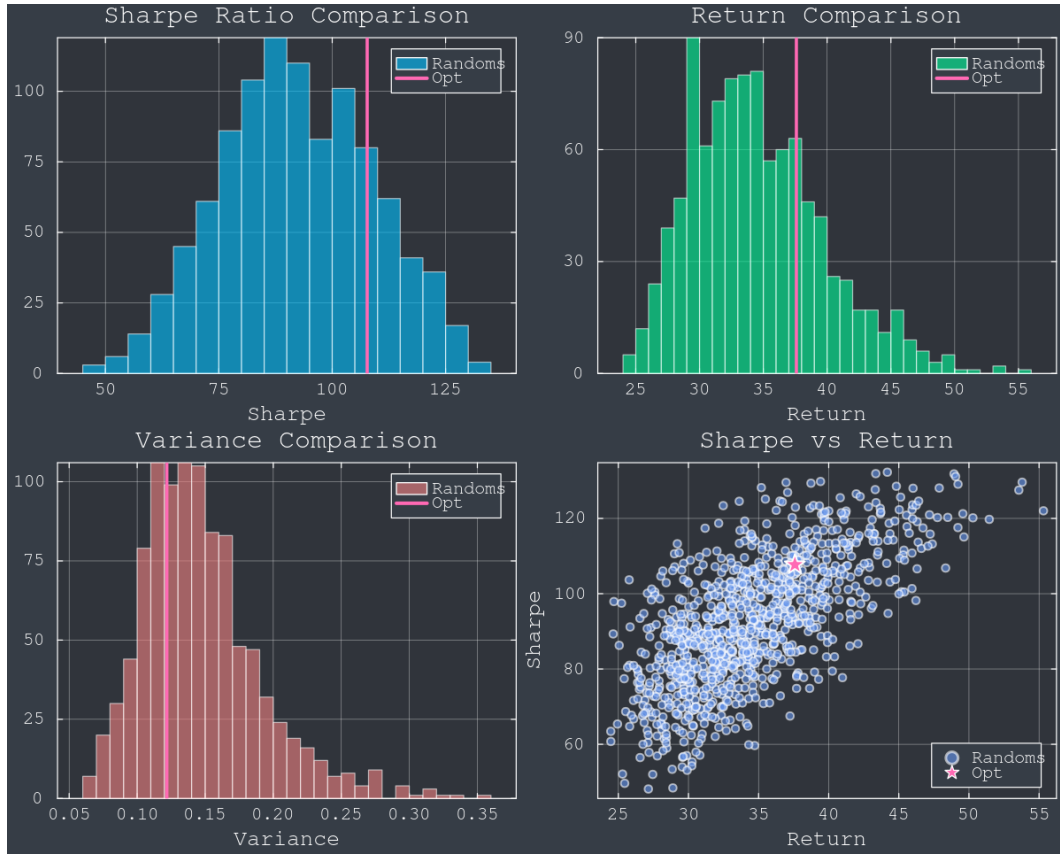


Fig. 3 Results for Out-of-Window Predictions

on Sharpe Ratio while also globally maximizing returns and minimizing variances (the two are not necessarily the same). However, Figure 3 shows that when the suggested portfolio is compared against true observed price trajectories, it regresses to the mean, with less extreme, albeit existent advantage. A natural follow up is whether the suggested portfolio actually has a nonzero expected value. This is what the results in Multi-Lineup Categorical attempt to show.

2. Multi-Lineup Categorical

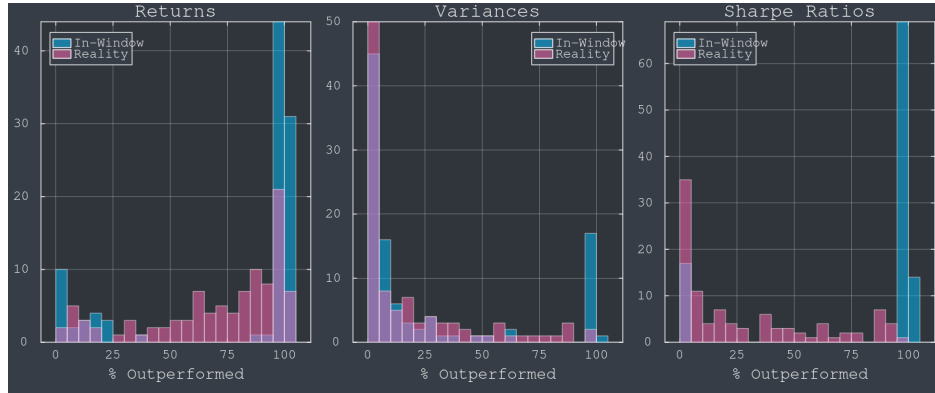


Fig. 4 Results for Out-of-Window Predictions on 100 random lineups of 10 blue-chip stocks

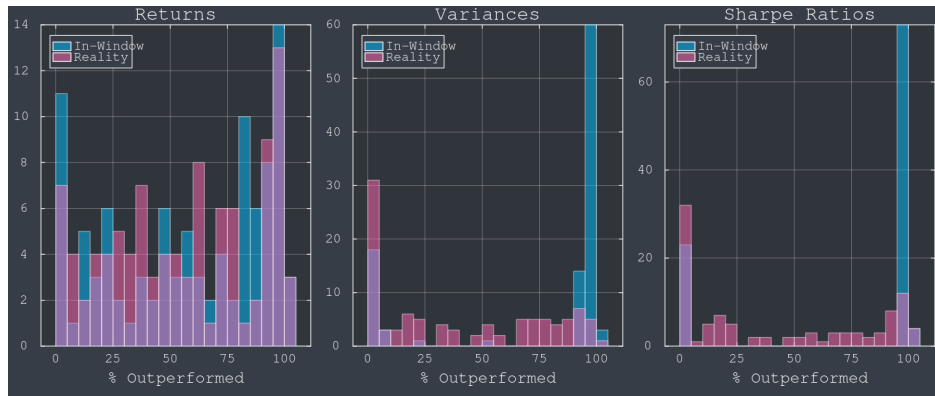


Fig. 5 Results for Out-of-Window Predictions on 100 random lineups of 10 etfs

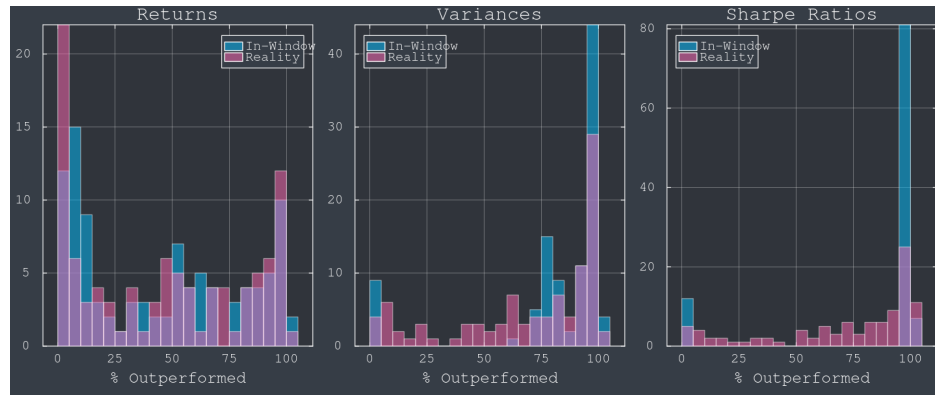


Fig. 6 Results for Out-of-Window Predictions on 100 random lineups of 10 bonds stocks